P2P Pokémon Battle Protocol (PokeProtocol) over UDP Rubric

CSNETWK Machine Problem 1T2526

Objective

This rubric evaluates a student's implementation of the P2P Pokémon Battle Protocol (PokeProtocol), with a total score of 125 points plus a 15-point bonus.

Deliverables

Source code + README

Deadline and Demo

13th and 14th week (to be announced by your instructor)

AI Usage Policy

Students may use AI tools (e.g., ChatGPT, Copilot) to help generate parts of the codebase or understand protocol requirements. However, all AI-generated code must be reviewed, tested, and verified by the student. The final submission should reflect the student's own understanding and ability to explain how the code works. Blindly copying output without testing or comprehension is not allowed. Submissions that show a lack of comprehension or appear to be largely untested AI output may receive reduced credit up to a grade of zero.

Groupings

Each group should have up to four members. All members of a group are expected to contribute meaningfully to the design, implementation, and testing of the project. Each participant must be capable of explaining all parts of the submission, even components they did not directly code. A detailed report of the tasks implemented by each team member should be recorded in the README or documentation. A sample of a Task Matrix is shown below. Additional tasks can be added or removed. In cases where uneven participation is suspected, instructors may request individual explanations. The group can drop any non-participating member in the submission.

Code sharing between different groups is strictly prohibited. While discussion of ideas, clarification, and protocol testing is encouraged, all source code must be the original work of the group submitting it. Any third-party libraries or tools used must be cited appropriately.

Violations of the collaboration and academic integrity policies—such as unauthorized sharing of code, plagiarism, or misrepresenting AI-generated work as entirely original—may result in a grade of zero for the project. In severe cases, such violations may also be referred to the appropriate academic disciplinary body for formal investigation. All students are expected to follow ethical guidelines and work within the permitted bounds of group work and tool usage.

To ensure fair contribution and collaborative integrity within groups, all members are expected to fulfill their assigned roles as outlined in the Work Distribution Matrix. If a member consistently fails to complete their assigned tasks, the group must first attempt to resolve the issue through clear, documented internal communication.

If the issue persists, the group is required to notify the course instructor or facilitator with supporting evidence (e.g., chat logs, Git commits, progress summaries). The instructor may then take one or more of the following actions:

- Individual grade adjustment based on documented contribution.
- Exclusion from the group if no remediation is possible

The remaining members must absorb the remaining work or redistribute with instructor approval. **False reporting of the distribution of work may be treated as academic misconduct**, resulting in a grade of zero or a disciplinary referral.

Checking

Project checking and demonstrations will be conducted with multiple groups simultaneously to assess interoperability, correctness, and compliance with the protocol specification. Each group must ensure their implementation functions correctly in a networked environment alongside others, as this setup reflects realistic use cases. Groups are expected to actively participate during checking and be able to explain and troubleshoot their contributions. Failure to perform during this stage may affect the final grade.

Activity	✅ Allowed	❌ Not Allowed
Working with a group (2–4 people)	Yes, group work is supported	Groups larger than 4
Using AI tools (e.g., ChatGPT, Copilot)	Yes, for generating, explaining, or debugging code	Submitting AI-generated code without review or testing
Discussing protocol ideas with others	Yes, general discussion of LSNP concepts is encouraged	Sharing source code between groups
Testing with another group	Yes, for interoperability testing and debugging	Submitting shared or merged code from different groups
Using open-source libraries/utilities	Yes, with proper citation	Using uncredited third-party code
Submitting identical code as another group	No	Plagiarism, even with minor changes
Individual understanding of group submission	Each member must understand the entire solution	Relying solely on one member or AI without team involvement

Submitting reused AI responses from others	No	Reusing AI content from another person's session
Acknowledging AI usage	Must include a note in README or code comments	Omitting any mention of AI assistance used

Grading Rubric (Total: 125/100 points)

Category	Criteria	Points	Description
Core Protocol Implementation	**UDP Sockets & Message Handling**	10	This criterion evaluates the correct implementation of UDP socket setup, binding, and the ability to send and receive datagrams efficiently.
	Handshake Process	10	This assesses the correct implementation of the handshake for Host, Joiner, and Spectator roles, including the secure exchange of the seed.
	Message Serialization	10	This checks if all protocol messages are correctly formatted into the specified key: value pairs, with proper newline delimiters.
Game Logic & State Management	**Turn-Based Flow**	15	This evaluates the flawless implementation of the 4-step turn flow and correct reversal of the turn order.
	Win/Loss Condition	5	This assesses whether the game correctly detects when a Pokémon faints and properly sends the GAME_OVER message.
	Battle State Synchronization	10	This checks that the battle state, including HP and status effects, remains perfectly synchronized on both peers throughout the battle.
Reliability & Error Handling	**Sequence Numbers & ACKs**	10	This evaluates the robust use of sequence numbers and acknowledgements for all critical battle messages to ensure reliable delivery.
	Retransmission Logic	10	This assesses the correct implementation of retransmission upon timeout, handling of the retry counter, and the ability to simulate packet loss for testing.
	Discrepancy Resolution	10	This checks for the correct detection of discrepancies in CALCULATION_REPORT and the

			initiation of the RESOLUTION_REQUEST process.
Features	**Damage Calculation**	10	This evaluates the accurate implementation of the damage formula, including separate Special Attack and Defense stats.
	Chat Functionality (Text)	5	This assesses the correct implementation of a plain-text chat feature that operates asynchronously without disrupting the battle state machine.
	Chat Functionality (Stickers)	5	This checks the correct handling and display of Base64 encoded sticker messages, and the ability to save the sticker data to a file.
	Verbose Mode	5	This evaluates the implementation of verbose mode, which should print all messages in a delimited fashion. When not in verbose mode, the app should hide all reliability and error handling messages, but still print error messages.
Code Quality & Design	**Readability & Comments**	5	This assesses the code's structure, readability, and the presence of clear, concise comments.
	Separation of Concerns	5	This checks for a well-structured codebase with clear separation between components like networking, game logic, and the UI.
Bonus	**Bonus Features**	15	This is for the implementation of additional features like a Spectator UI or a graphical battle interface.

AI Disclaimer

The protocol is primarily designed by the author. AI tools, primarily ChatGPT and CoPilot, are leveraged for the writing of the RFC. AI was leveraged to help formulate message structures and RFC formatting. Additionally, it supported the writing of the grading rubric, collaboration policy, and testing guidelines. All AI-generated content was thoroughly reviewed, validated, and adapted to meet the functional and educational goals of the project.